

Simulation d'un écosystème proie-prédateur

Durée : 3 h

Langage : Python 3

Travail : individuel ou binôme

1. Contexte

Vous êtes chargé de développer un programme permettant de simuler l'évolution d'un petit écosystème.

Cet écosystème est constitué principalement de deux populations :

- des **proies**, par exemple des lapins ;
- des **prédateurs**, par exemple des loups.

Chaque animal vit dans un environnement représenté par une grille.

Au cours de la simulation :

- les animaux se déplacent ;
- ils vieillissent ;
- ils consomment de l'énergie ;
- les prédateurs chassent les proies ;
- les proies peuvent se reproduire ;
- les prédateurs peuvent se reproduire ;
- les animaux trop faibles meurent.

L'objectif n'est pas de construire un modèle biologique réaliste. Il s'agit de construire un **modèle informatique suffisamment simple pour mettre en pratique les concepts de programmation orientée objet**.

2. Problématique

Vous devrez répondre progressivement à la question suivante :

Comment concevoir un programme orienté objet capable de représenter et de simuler un écosystème dans lequel différents types d'animaux interagissent ?

Au cours du projet, vous découvrirez ou mettrez en pratique :

- les classes ;
- les objets ;
- les attributs ;
- les méthodes ;
- les constructeurs ;
- l'encapsulation ;
- l'héritage ;
- l'abstraction ;
- le polymorphisme ;
- la composition ;
- les collections d'objets ;
- les interactions entre objets ;
- les tests unitaires ;
- la modélisation UML.

3. Organisation du projet

À la fin du TP, votre projet devra avoir une organisation proche de celle-ci :

```
simulation_ecosysteme/  
├── animal.py  
├── proie.py  
├── predateur.py  
├── lapin.py  
├── loup.py  
├── environnement.py  
├── simulation.py  
├── test_animal.py  
├── test_environnement.py  
└── main.py
```

Vous êtes libre d'ajouter d'autres fichiers si cela améliore l'organisation du programme.

4. Règles générales de la simulation

L'environnement possède :

- une largeur ;
- une hauteur ;
- une collection d'animaux.

Chaque animal possède :

- une position ;
- une énergie ;

- un âge ;
- une vitesse.

Une simulation est composée d'une succession de **tours**.

À chaque tour, les animaux doivent notamment :

1. se déplacer ;
2. vieillir ;
3. perdre de l'énergie ;
4. interagir avec les autres animaux ;
5. éventuellement se reproduire ;
6. mourir si leur énergie devient insuffisante.

Les règles précises seront introduites progressivement.

Séance 1 — Premiers objets

Mission

Nous allons commencer avec une seule espèce : le lapin.

Avant d'utiliser l'héritage ou le polymorphisme, vous allez apprendre à représenter un élément du monde réel avec une classe Python.

Exercice 1 — Identifier les caractéristiques d'un lapin

Un lapin doit être caractérisé par :

- sa position horizontale x ;
- sa position verticale y ;
- son énergie ;
- son âge ;
- sa vitesse.

Répondez aux questions suivantes.

Question 1

Quels éléments doivent être représentés par des **attributs** ?

Question 2

Quelles actions peut réaliser un lapin ?

Proposez au moins trois méthodes.

Question 3

Représentez votre classe sous forme UML.

Exercice 2 — Première classe

Créez une classe `Lapin`.

Le constructeur doit recevoir les coordonnées initiales :

```
lapin = Lapin(10, 20)
```

Le lapin doit avoir par défaut :

```
énergie = 100
âge = 0
vitesse = 1
```

Exemple de structure :

```
class Lapin:
    def __init__(self, x, y):
        # À compléter
        pass
```

Exercice 3 — Déplacement

Ajoutez une méthode :

```
se_deplacer(dx, dy)
```

Elle doit modifier la position du lapin.

Exemple :

```
lapin = Lapin(10, 20)
lapin.se_deplacer(3, -2)

print(lapin.x)
print(lapin.y)
```

Résultat attendu :

```
13
18
```

Exercice 4 — Vieillessement

Ajoutez :

```
vieillir()
```

À chaque appel :

- l'âge augmente de 1 ;
- l'énergie diminue de 1.

Exemple :

```
lapin.vieillir()
```

Exercice 5 — Mort

Ajoutez :

```
est_vivant()
```

La méthode doit retourner :

True

si l'énergie est strictement positive et :

False

sinon.

Exercice 6 — Plusieurs objets

Créez cinq lapins et placez-les dans une liste :

```
lapins = [  
    Lapin(10, 10),  
    Lapin(20, 30),  
    Lapin(50, 20),  
    Lapin(70, 40),  
    Lapin(80, 80)  
]
```

Faites vieillir tous les lapins avec une boucle.

Question

Pourquoi une liste d'objets est-elle plus intéressante ici que cinq variables distinctes ?

Séance 2 — Héritage

Mission

Nous voulons maintenant ajouter des loups.

Un loup possède lui aussi :

- une position ;
- une énergie ;
- un âge ;
- une vitesse.

Il peut également :

- vieillir ;
- se déplacer ;
- mourir.

Vous constatez donc une duplication du code.

Exercice 7 — Identifier la duplication

Comparez :

```
Lapin
- x
- y
- énergie
- âge
- vitesse
- vieillir()
- est_vivant()
```

et :

```
Loup
- x
- y
- énergie
- âge
- vitesse
```

- vieillir()
- est_vivant()

Question

Quel concept de POO permet de factoriser ces caractéristiques communes ?

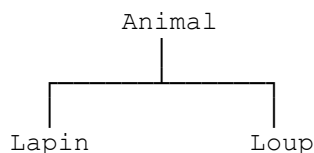
Exercice 8 — Classe Animal

Créez :

```
class Animal:
    ...
```

Déplacez dans cette classe les attributs et méthodes communs.

Vous devez obtenir une structure conceptuelle proche de :



Exercice 9 — Hériter d'Animal

Modifiez vos classes :

```
class Lapin(Animal):
    ...
```

et :

```
class Loup(Animal):
    ...
```

Utilisez `super()` lorsque cela est pertinent.

Exercice 10 — Comportements spécifiques

Ajoutez :

Pour le lapin :

```
fuir()
```

Pour le loup :

```
chasser()
```

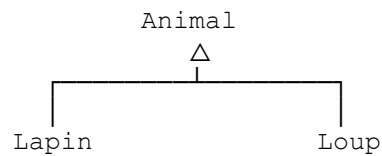
Pour le moment, ces méthodes peuvent simplement afficher un message.

Exemple :

```
Le lapin fuit !  
Le loup chasse !
```

Question UML

Complétez le diagramme :



Ajoutez les principaux attributs et méthodes.

Séance 3 — Polymorphisme

Mission

L'environnement doit pouvoir manipuler indifféremment des lapins et des loups.

Nous voulons pouvoir écrire :

```
animaux = [  
    Lapin(10, 10),  
    Loup(20, 20),  
    Lapin(30, 30),  
    Loup(40, 40)  
]  
  
for animal in animaux:  
    animal.se_deplacer()
```

Le programme doit appeler le comportement approprié.

Exercice 11 — Une méthode commune

Ajoutez dans `Animal` :

```
se_deplacer()
```

Puis faites en sorte que `Lapin` et `Loup` possèdent chacun leur propre implémentation.

Par exemple :

```
Lapin → déplacement de 1 case  
Loup  → déplacement de 2 cases
```

Exercice 12 — Polymorphisme

Testez :

```
for animal in animaux:  
    animal.se_deplacer()
```

Question

Pourquoi n'avons-nous pas besoin d'écrire :

```
if isinstance(animal, Lapin):  
    ...  
elif isinstance(animal, Loup):  
    ...
```

?

Expliquez avec vos propres mots ce qu'est le **polymorphisme**.

Exercice 13 — Vers une classe abstraite

Une question se pose :

Que signifie réellement `Animal.se_deplacer()` ?

Un animal générique n'a pas nécessairement un comportement de déplacement précis.

Nous allons donc rendre cette méthode abstraite.

Utilisez :

```
from abc import ABC, abstractmethod
```

et transformez `Animal` en classe abstraite.

Exercice 14 — Tester l'abstraction

Essayez :

```
animal = Animal(10, 10)
```

Que se passe-t-il ?

Expliquez pourquoi.

Séance 4 — Encapsulation et interactions

Mission

Nous allons maintenant nous intéresser aux données internes des objets.

Actuellement, un programme extérieur pourrait faire :

```
lapin.energie = -500
```

Cela n'est pas souhaitable.

Exercice 15 — Encapsuler l'énergie

Modifiez la classe `Animal` afin que l'énergie soit gérée par l'objet lui-même.

Ajoutez par exemple :

```
perdre_energie(quantite)
```

et :

```
gagner_energie(quantite)
```

Réfléchissez à la manière de garantir que l'énergie ne devienne pas incohérente.

Exercice 16 — Chasse

Un loup doit pouvoir manger un lapin lorsque les deux animaux occupent la même position.

Créez une méthode permettant de réaliser cette interaction.

Exemple conceptuel :

```
loup.chasser(lapin)
```

Si les positions sont identiques :

- le lapin meurt ;
- le loup récupère de l'énergie.

Sinon, rien ne se passe.

Exercice 17 — Distance

Ajoutez une fonction ou une méthode permettant de calculer la distance entre deux animaux.

Vous pouvez utiliser :

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Utilisez le module `math`.

Question de conception

Cette fonctionnalité doit-elle appartenir :

- à `Animal` ?
- à `Environnement` ?
- à une fonction indépendante ?

Justifiez votre choix.

Exercice 18 — Détection d'une proie

Modifiez le comportement du loup.

Le loup doit rechercher une proie située à proximité.

Vous pouvez choisir un rayon de détection, par exemple :

```
rayon = 10
```

Le loup peut alors :

1. rechercher les proies proches ;
 2. choisir une proie ;
 3. se déplacer vers elle ;
 4. l'attaquer lorsqu'il arrive à proximité.
-

Séance 5 — Composition et environnement

Mission

Nous avons maintenant plusieurs objets.

Il faut construire un objet capable de gérer l'ensemble de l'écosystème.

Nous allons créer :

Environnement

Exercice 19 — Classe Environnement

Créez :

```
class Environnement:
    def __init__(self, largeur, hauteur):
        ...
```

L'environnement doit contenir :

```
largeur
hauteur
animaux
```

Exercice 20 — Ajouter des animaux

Ajoutez :

```
ajouter(animal)
```

Exemple :

```
env = Environnement(100, 100)
env.ajouter(Lapin(10, 20))
```

```
env.ajouter(Loup(80, 20))
```

Question

Quel concept de POO illustre le fait qu'un environnement **contient** des animaux ?

Exercice 21 — Supprimer les morts

Ajoutez :

```
supprimer_morts()
```

Cette méthode doit supprimer les animaux dont :

```
animal.est_vivant()
```

retourne `False`.

Exercice 22 — Un tour de simulation

Ajoutez :

```
simuler_un_tour()
```

Pour chaque animal :

1. le faire se déplacer ;
2. le faire vieillir ;
3. gérer sa consommation d'énergie.

Puis supprimer les animaux morts.

Exercice 23 — Reproduction

Ajoutez une mécanique de reproduction.

Un animal peut se reproduire si :

```
âge >= 5  
ET  
énergie >= 60
```

Vous devrez réfléchir à la méthode :

```
peut_se_reproduire()
```

et à :

```
reproduire()
```

La reproduction doit produire un nouvel animal.

Question de conception

La méthode `reproduire()` doit-elle être définie dans :

- `Animal` ?
- `Proie` ?
- `Lapin` ?
- une autre classe ?

Justifiez votre choix.

Séance 6 — Projet final

Mission générale

Vous allez maintenant intégrer les différents éléments pour construire une simulation complète.

5. Cahier des charges final

Votre programme doit comporter au minimum les classes suivantes :

```
Animal  
Proie  
Predateur  
Lapin  
Loup  
Environnement  
Simulation
```

La hiérarchie devra être cohérente.

Vous devrez notamment utiliser :

- une classe abstraite ;

- l'héritage ;
 - le polymorphisme ;
 - l'encapsulation ;
 - la composition.
-

6. Règles minimales

Animal

Un animal possède :

- une position ;
- une énergie ;
- un âge ;
- une vitesse.

Il sait :

- se déplacer ;
 - vieillir ;
 - perdre de l'énergie ;
 - vérifier s'il est vivant.
-

Proie

Une proie :

- se déplace ;
 - peut fuir ;
 - peut se nourrir ;
 - peut se reproduire.
-

Prédateur

Un prédateur :

- se déplace ;
- recherche des proies ;
- peut chasser ;
- récupère de l'énergie après une chasse ;
- peut se reproduire.

Environnement

L'environnement :

- possède une largeur ;
- possède une hauteur ;
- contient les animaux ;
- ajoute des animaux ;
- supprime les animaux morts ;
- exécute un tour de simulation.

Simulation

La simulation :

- crée l'environnement ;
- initialise les populations ;
- exécute les tours ;
- affiche les statistiques.

7. Statistiques

À chaque tour, affichez au minimum :

```
Tour : 25
Proies : 73
Prédateurs : 12
Total : 85
```

Vous pouvez ajouter :

- énergie moyenne ;
- âge moyen ;
- nombre de naissances ;
- nombre de morts ;
- nombre de proies mangées.

8. UML final

Réalisez un diagramme UML représentant votre conception.

Il devra notamment faire apparaître :

- les classes ;
- les attributs principaux ;
- les méthodes principales ;
- les relations d'héritage ;
- la composition entre `Environnement` et les animaux.

Vous devez être capable d'expliquer oralement votre diagramme.

9. Tests unitaires

Vous devez créer au minimum les tests suivants.

Test 1 — Création d'un animal

Vérifier :

```
âge = 0
énergie = 100
```

Test 2 — Vieillesse

Après un tour :

```
âge augmente
énergie diminue
```

Test 3 — Mort

Un animal ayant :

```
énergie = 0
```

doit être considéré comme mort.

Test 4 — Déplacement

Vérifier que les coordonnées changent correctement.

Test 5 — Chasse

Créer un loup et un lapin à la même position.

Vérifier que :

- le lapin est mort ;
- le loup récupère de l'énergie.

Test 6 — Environnement

Ajouter plusieurs animaux et vérifier que :

```
len(environnement.animaux)
```

correspond au nombre attendu.

Test 7 — Suppression

Ajouter un animal mort et vérifier qu'il est supprimé.

10. Utilisation de unittest

Exemple :

```
import unittest

from lapin import Lapin

class TestLapin(unittest.TestCase):

    def test_creation(self):
        lapin = Lapin(10, 20)

        self.assertEqual(lapin.age, 0)
        self.assertEqual(lapin.energie, 100)

    def test_vieillissement(self):
        lapin = Lapin(10, 20)

        lapin.vieillir()

        self.assertEqual(lapin.age, 1)
        self.assertEqual(lapin.energie, 99)

if __name__ == "__main__":
    unittest.main()
```

Lancez les tests avec :

```
python -m unittest
```

11. Critères de qualité

Votre programme sera évalué sur son fonctionnement mais également sur sa conception.

Un programme qui fonctionne mais qui contient beaucoup de duplication ou qui mélange toutes les responsabilités ne sera pas considéré comme une bonne solution objet.

Vous devez notamment vous demander :

- Une classe a-t-elle une responsabilité claire ?
 - Le code est-il réutilisable ?
 - Les classes sont-elles suffisamment indépendantes ?
 - Utilisez-vous correctement l'héritage ?
 - Le polymorphisme évite-t-il des `if` inutiles ?
 - Les données internes sont-elles correctement protégées ?
 - Les méthodes sont-elles courtes et compréhensibles ?
-

12. Défis facultatifs

Défi 1 — Ressources

Ajoutez une classe :

```
Ressource
```

Les proies peuvent manger les ressources.

Défi 2 — Plusieurs espèces

Ajoutez :

```
Renard  
Cerf  
Aigle
```

Chaque espèce doit avoir un comportement différent.

Défi 3 — Terrain

Ajoutez plusieurs types de terrain :

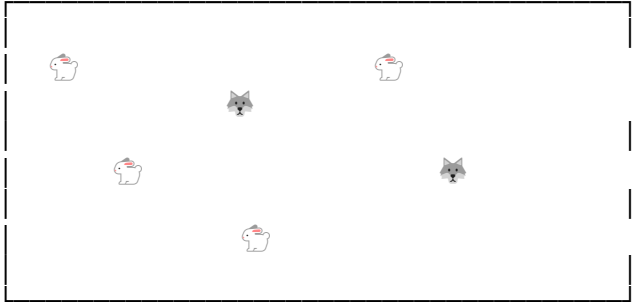
```
Plaine  
Forêt  
Eau  
Montagne
```

Certains animaux peuvent être favorisés ou défavorisés selon le terrain.

Défi 4 — Visualisation

Créez une interface graphique simple avec `tkinter`.

Par exemple :



```
Tour : 150
Proies : 42
Prédateurs : 7
```

13. Questions finales

Répondez en quelques lignes.

Q1

Quelle différence faites-vous entre une classe et un objet ?

Q2

Pourquoi `Lapin` peut-il hériter de `Proie` ?

Q3

Pourquoi `Loup` peut-il hériter de `Predateur` ?

Q4

Donnez un exemple d'héritage dans votre programme.

Q5

Donnez un exemple de composition.

Q6

Donnez un exemple de polymorphisme.

Q7

Pourquoi `Animal` peut-il être abstrait ?

Q8

Pourquoi est-il préférable de demander à un objet de modifier son énergie plutôt que de modifier directement son attribut depuis l'extérieur ?

Q9

Que se passerait-il si nous supprimions la classe `Environnement` et mettions toute la logique dans `main.py` ?

Q10

Quel principe de conception avez-vous trouvé le plus difficile à comprendre ? Expliquez pourquoi.

14. Livrables

À la fin du TP, vous devez remettre :

1. le code Python complet ;
2. le diagramme UML ;
3. les tests unitaires ;
4. un court rapport de conception ;
5. une démonstration de la simulation.

Le rapport devra notamment expliquer les choix concernant :

- l'héritage ;
- le polymorphisme ;
- l'encapsulation ;
- la composition ;
- la répartition des responsabilités entre les classes.

En début de séance prochaine, vous ferez une soutenance de 10 minutes par binôme : vous expliquerez le diagramme UML, vous justifierez un choix d'héritage, vous montrerez un exemple de polymorphisme et exécuterez leurs tests.